

Smart City Zero-Disk IoT Infrastructure

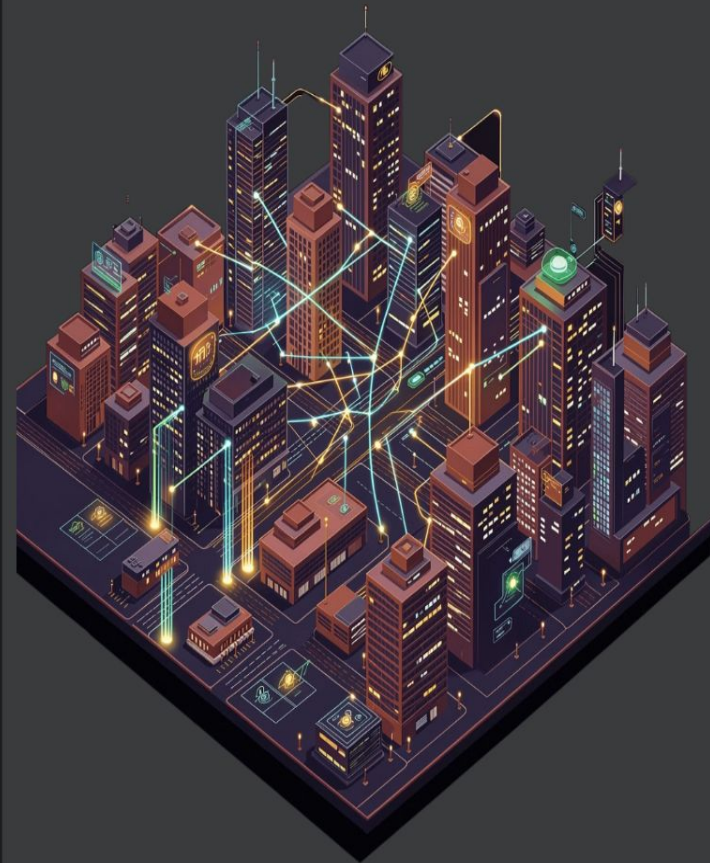
A scalable, cost-optimized platform for ingesting and processing massive urban sensor data with **zero persistent disk** in the ingestion path.

- [Prathamesh Abhang](#)
- [Nargiza Deidieva](#)
- [Sai Lakshmi Priya Kothacheruvu](#)
- [Tanishq Kureel](#)

MSDS 432 - Foundations of Data Engineering

Northwestern University | Group 4 | Project 4

Phase 2 - Detailed Design Presentation



Reimagining IoT Storage at Scale



Urban IoT deployments generate **terabytes of sensor data daily**. Traditional architectures persist every event to disk — a costly bottleneck.

This project builds a **Zero-Disk ingestion pipeline** that buffers data in memory, streams it to object storage, and reduces storage costs by up to **90%**.

90% Cost Reduction

Real-Time Analytics

Zero Disk Writes

PROBLEM STATEMENT

The IoT Data Deluge

Volume

Millions of events per second from air quality, traffic, and environmental sensors across a smart city.

Velocity

High-frequency ingestion demands sub-millisecond processing — disk I/O cannot keep pace.

Cost

Persistent storage for raw IoT data is prohibitively expensive at city scale.

Reliability

Uncontrolled ingestion rates cause cascading failures and data loss under load.



OBJECTIVE

What We're Building



Golang Ingestion Service

Lightweight, concurrent microservice handling validation, transformation, and routing of sensor events.



Real-Time Analytics

In-memory aggregation and streaming metrics exposed via Prometheus for live dashboards.



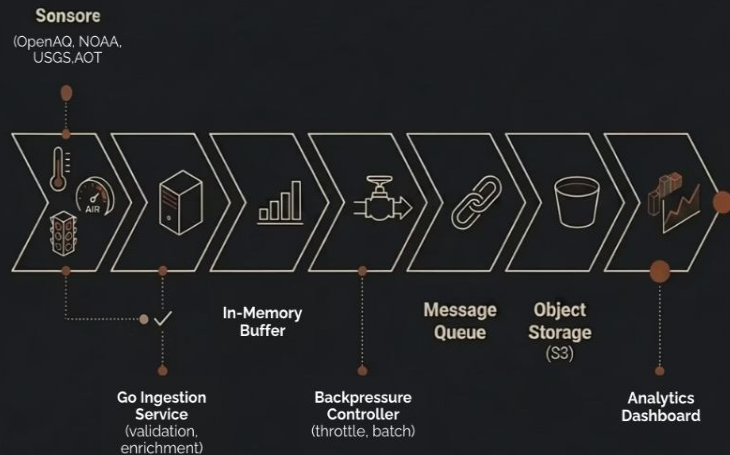
Cost Optimization

Batch writes to S3-compatible object storage eliminate per-event disk writes, cutting costs by 90%.

End-to-End Data Pipeline

Data flows from city sensors through a fully in-memory pipeline before reaching durable object storage — *no disk writes in the critical path*.

- **Sensors** publish events via REST HTTP API
- **Go Service** validates and enriches payloads
- **Buffer + Back-pressure** controls flow under load
- **Queue** decouples ingestion from storage
- **Object Storage** persists batched data
- **Dashboard** visualizes real-time metrics



City-Scale Sensor Feeds



OpenAQ & NOAA

Real-time air quality (PM2.5, NO2, CO) from OpenAQ and weather context (temperature, humidity, wind speed) from NOAA weather directly affects sensor readings and pollution dispersion.

Chicago Traffic Data

Vehicle counts, speed, and congestion metrics from the Chicago Data Portal open feeds correlated with air quality for vehicle emission analysis.

Array of Things

Hyperlocal urban sensor data, sound, light, vibration from Chicago's AoT network.

Go Sensor Simulator

Custom-built Go program generating synthetic sensor readings at configurable rates, used for stress-testing backpressure and filling data gaps during API downtime.

Incoming Event Payload

Sample JSON Event

• Each sensor publishes a lightweight JSON payload to the system • The Go service validates three critical aspects before accepting events:

- Schema validation (correct structure)
- Timestamps (proper format and reasonable values)
- Value ranges (readings fall within expected parameters) • Walk through the sample payload structure:
- Sensor ID identifies the specific device
- ISO 8601 timestamp for precise event timing
- Location coordinates (latitude/longitude)
- Multiple readings in one payload (temperature, humidity, PM2.5, CO2)
- Source field tracks data origin • This lightweight approach keeps network traffic minimal while the validation layer ensures data quality upstream

```
{
  "sensor_id": "chi-aq-017",
  "timestamp": "2024-01-15T14:32:01Z",
  "location": {
    "lat": 41.8781,
    "lon": -87.6298
  },
  "readings": {
    "temperature_c": 18.4,
    "humidity_pct": 62,
    "pm25": 12.3,
    "co2_ppm": 418
  },
  "source": "openaq"
}
```

The Ingestion Engine

10,000+

Events buffered per second via Go channels

4 Components

HTTP Poller · Back-pressure Manager · Pub/Sub Queue · Async Writer

Zero Blocking

Goroutines decouple ingestion speed from storage write speed

HTTP Ingestion

Accepts events over REST normalizing payloads into a unified internal schema.

Schema Validation

JSON Schema enforcement with range checks on numeric sensor values before queuing.

Grafana Observability

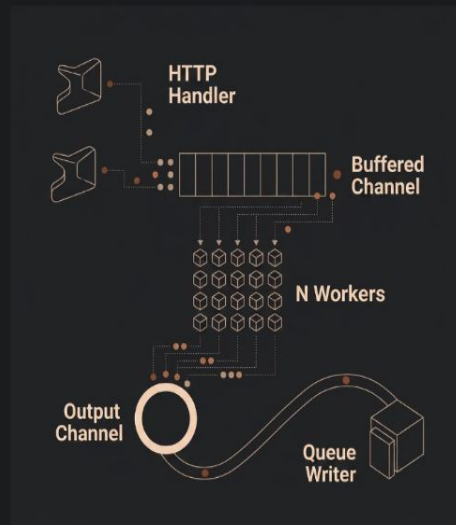
Exposes `events_ingested_total`, `validation_errors`, and `buffer_depth` - feeding live Grafana dashboards via TimescaleDB.

Graceful Shutdown

Drains in-flight events on SIGTERM before closing connections, zero data loss on deploy.

Goroutines, Channels & Worker Pools

Go's concurrency primitives handle millions of events without thread-per-request overhead. A fixed **worker pool** bounds resource usage under load.



01

HTTP Handler

Spawns a goroutine per request, serializing JSON into a channel.

02

Buffered Channel

Decouples producers from workers with a bounded ring buffer.

03

Worker Pool

Fixed N workers consume from the channel, validating and batching.

04

Output Channel

Validated batches flow to the queue writer goroutine.

Staying Stable Under Load



Rate Limiting

Token bucket algorithm caps per-sensor ingestion rate, preventing any single source from saturating the pipeline.



Adaptive Batching

Events are coalesced into batches — larger under low load, smaller under high latency — optimizing S3 write efficiency.



Exponential Retries

Failed writes to the queue use jittered exponential backoff, avoiding thundering-herd recovery spikes.



Circuit Breaker

When buffer depth exceeds a threshold, the circuit opens — rejecting non-critical events while preserving system stability.

Back-pressure Strategy

Trigger	Go channel reaches 80% capacity
Action	Drop policy activates, low-priority duplicates removed
Safety Net	GCP Pub/Sub holds messages for up to 7 days
Recovery	Exponential backoff resumes writes when storage is available

Dataset Overview - OpenAQ Multi-Parameter Air Quality Data

Clinton C403/C304/AH113 Monitoring Station, Chicago, USA - Multi-Parameter Analysis

Total Records

2378

Active Sensors

3

Outliers (IQR)

78

Duplicates

0

Data Source

Real-world air quality data pulled via OpenAQ API v3. Location: Clinton C403/C304/AH113, Chicago, USA. Multi-parameter analysis covering O₃, NO₂, and CO measurements across 3 active sensors. The OpenAQ API is one of the project's primary data sources - this confirms the pipeline can ingest live global sensor feeds regardless of geography, validating the Zero-Disk architecture's location-agnostic design.

★★★★★ 2,378

Total sensor records collected

★★★★☆ 3.28%

Outlier rate (IQR method)

★★★★☆ 3

Active sensors monitored

★★★★★ 60 min

Consistent reporting interval

O₃, NO₂, CO (ppm)

Parameter measured

2016-2018

Data coverage window

0%

Missing values — clean dataset,
only location data was missing

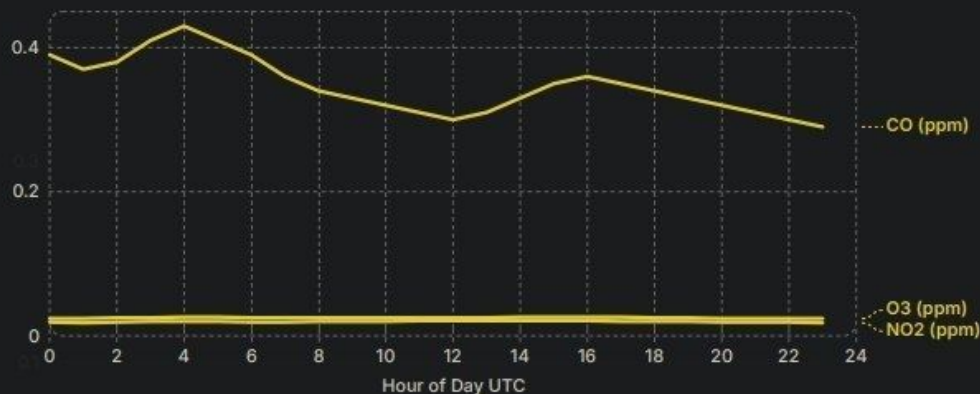
0

Duplicate rows found

Multi-Parameter Sensor Behavior - CO, NO2, and O3 Patterns

2,378 hourly readings from Clinton, Chicago - three pollutants analyzed

24-Hour Diurnal Pattern (Avg. by Hour UTC)



Key Findings

- Total records: 2,378 across 3 sensors and 3 parameters
- Parameters: CO, NO2, O3 (ppm)
- 78 outliers detected at 3.28% rate - IQR method
- Measurement interval: exactly 60.0 minutes - consistent across all readings
- Latitude/longitude: 100% missing - exact location data was not available in this dataset
- CO shows strongest diurnal cycle; NO2 and O3 relatively stable

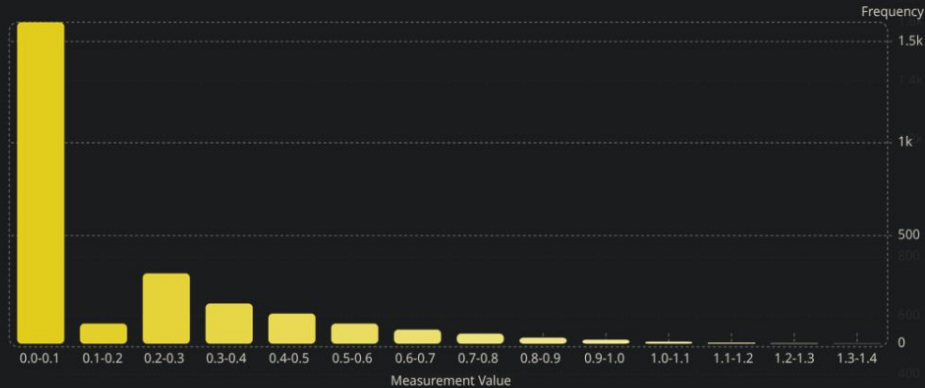
Peak: Hour 04 for CO at ~0.43 ppm · NO2/O3: relatively flat across the day

- CO Peak: Strong diurnal pattern with peak at Hour 04
- NO2 & O3: Remain relatively flat throughout the day
- CO Variation: Peak-to-trough variation highest in CO measurements

Data Distribution & Outlier Detection

Measurement value distribution and IQR-based outlier identification across all parameters

Distribution of All Sensor Measurement Values



This histogram shows the data is heavily skewed toward low values with a long tail.

- Lower bound: -0.27 (IQR method)
- Upper bound: 0.482 (IQR method)
- Number of outliers: 78
- Outlier percentage: 3.28%
- Data heavily concentrated in 0.0-0.3 range with 78 extreme values flagged for review

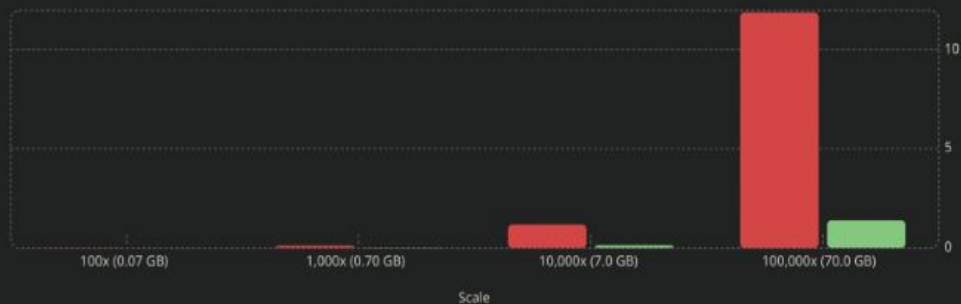
Outlier Detection Summary

- IQR Method: $Q1 = 0.05$, $Q3 = 0.15$, $IQR = 0.10$
- Lower bound: $Q1 - 1.5 \times IQR = -0.27$
- Upper bound: $Q3 + 1.5 \times IQR = 0.482$
- Any values beyond these bounds flagged as outliers
- 78 measurements exceed upper bound (0.482)
- Outliers represent extreme pollution events or sensor anomalies
- All outliers retained for analysis - not removed

Zero-Disk Cost Savings - Proven by Real Data

Based on actual OpenAQ sensor data — scaled using real 2026 GCP pricing

SSD Cost (\$/mo) GCS Cost (\$/mo)



88.2% Cost Reduction

Validated on real sensor data — 2026 GCP pricing

GCS Standard storage at \$0.020/GB/month versus SSD-backed managed database at \$0.170/GB/month. At city-wide scale with hundreds of sensors reporting hourly, annual savings compound significantly.

88.2% saved

\$0.020/GB

GCS Standard rate (us-central1, 2026)

\$0.170/GB

SSD managed DB baseline (2026)

Scale	Dataset	SSD Cost/mo	GCS Cost/mo	Saving
100x	0.07 GB	\$0.012	\$0.0014	88.2%
1,000x	0.70 GB	\$0.119	\$0.014	88.2%
10,000x	7.0 GB	\$1.19	\$0.140	88.2%
100,000x	70.0 GB citywide	\$11.90	\$1.40	88.2%

Storage cost is the single biggest operational expense in urban IoT deployments - Zero-Disk architecture eliminates it.

Data Sources

Three ingestion sources feed the pipeline — two live, one synthetic for performance validation.



OpenAQ API

Live air quality data from global monitoring stations via REST API.



Array of Things

Chicago urban sensor network — environment, traffic, and infrastructure telemetry.



Go Sensor Simulator

Synthetic load generator for throughput testing and backpressure validation.

Go Ingestion Microservice

All core ingestion logic is implemented in **Golang** - concurrency, back-pressure, and orchestration in a single stateless service.



Ingestion

Unified entry point for all sensor data streams



Concurrency

Goroutines handle sources independently

Goroutines

One lightweight goroutine spawned per data source — no thread-per-request overhead

10,000

Buffered channel capacity before backpressure policy activates



Backpressure

Channels + select prevent overload



Orchestration

Coordinates flow to storage layers

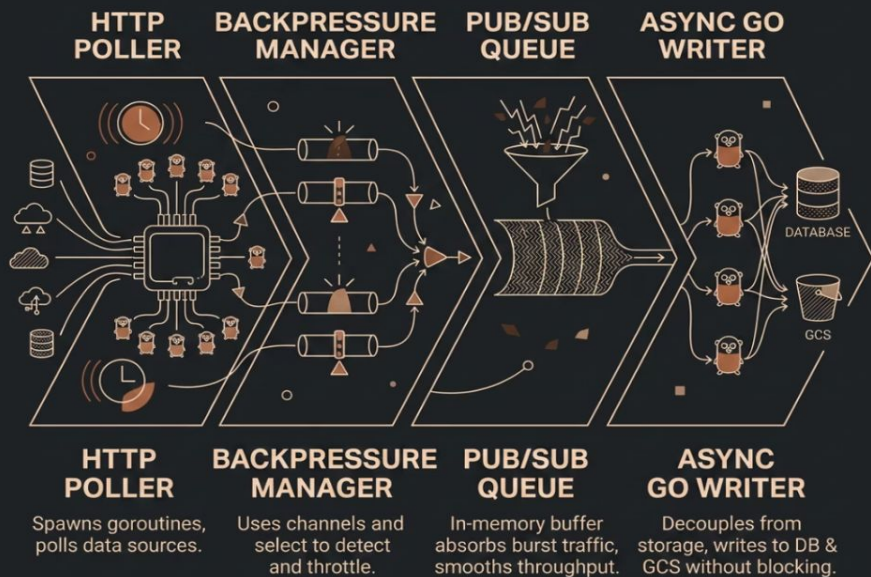
Async

Writer decouples ingestion speed from storage write speed completely

Stateless

Single deployable Go binary — no local state, runs identically in Docker and GKE

Ingestion Microservice — Internal Components



Four Components, One Service

The ingestion microservice is composed of four tightly integrated components, each addressing a specific challenge in high-throughput IoT data collection.

Together they ensure the system remains **resilient under burst load** — no data loss, no blocking, no single point of failure.

Hot Storage — Recent Data



TimescaleDB

A PostgreSQL extension optimized for time-series workloads. Stores the **last 72 hours** of sensor data, enabling sub-second queries for live dashboards.

- Automatic partitioning by time interval
- Compression reduces storage footprint
- Direct integration with Grafana
- SQL-compatible — no new query language

Cold Storage - GCS + Parquet

After 72 hours, data is flushed to **Google Cloud Storage** as columnar Parquet files; no local disk, no SSD costs.

~90% Cost Savings

GCS: ~\$0.02/GB/month vs SSD-backed DB: ~\$0.17/GB/month. For 1 TB of data: \$20/month vs \$174/month - saving over \$150/month or 88% cost reduction.

Zero Local Disk

All persistent data lives in cloud object storage

Parquet Format

Columnar, compressed, BigQuery-native

Query & Analytics — Dual Paths

Workloads are **separated by design** — real-time and historical queries never compete for resources.

- **Hot path:** TimescaleDB → Grafana for live monitoring
- **Cold path:** BigQuery → Streamlit for EDA and cost analysis
- Each path optimized for its latency and cost profile



Metadata Store

A dedicated **PostgreSQL** instance holds all system metadata separate from time-series data for clean separation of concerns.

Sensor Registry

Device ID, location, type, and status for every connected sensor.

Device Configuration

Polling intervals, API endpoints, and active/inactive flags per device.

Cost Metrics Tracking

Records GCS vs. SSD cost comparisons, quantifying the zero-disk savings.

Architecture Summary

7 Layers. One Cohesive System.

01

Data Sources

OpenAQ, Array of Things, Go Simulator

03

Hot Storage

TimescaleDB, 72-hour retention, real-time queries

05

Query & Analytics

TimescaleDB for live, BigQuery for historical

07

Orchestration

Docker Compose + GKE Autopilot + K8s Sidecar

02

Go Ingestion Microservice

Goroutines, backpressure, Pub/Sub

04

Cold Storage

GCS + Parquet, zero disk, 90% cost saving

06

Metadata Store

PostgreSQL sensor registry and cost tracking

08

Dashboards

Grafana for real-time, Streamlit for EDA and cost reports

Thank You

Smart City Zero-Disk IoT Infrastructure

Zero disk. Real-time insights. 90% cost savings.



Q&A

Team

Prathamesh Abhang

Nargiza Deidieva

Sai Lakshmi Priya Kothacheruvu

Tanishq Kureel

We built this on

- Golang: Core ingestion and backpressure
- TimescaleDB + GCS: Hot and cold storage
- GKE + Docker: Orchestration
- Grafana + Streamlit: Dashboards